```
# ====================================
# Import libraries
# ====================================
import torch
from torch import nn, optim
import torch.nn.functional as F
from torch.utils.data import random_split, DataLoader
import torchvision.transforms.v2 as T
from torch.utils.data import TensorDataset
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random

#Nb. I have tried to re-use the code for the classification task as much
#as I could for consistency. I have flagged the spots where something changes with "NEW".
```

```
#Check whether a GPU is available
print("GPU available:", torch.cuda.is_available())

#Check which GPU is being used
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
else:
    print("Using CPU")
```

```
GPU available: True
GPU name: Tesla T4
```

```
# ====================================
# Reproducibility and device
# ====================================
r_seed = 99
generator = torch.Generator()

def reset_seed(seed=r_seed):
    """Reset all seeds at once."""
    random.seed(seed) #reset python random seed
    np.random.seed(seed) #reset numpy random seed
    torch.manual_seed(seed) #reset pytorch random seed
    generator.manual_seed(seed) #reset the dataloader random seed
    if torch.cuda.is_available(): #if we use the gpu
        torch.cuda.manual_seed_all(seed)

reset_seed()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
#This cell runs on Google Colab if you save the data files in a folder called "data" in Drive.
#To run locally, skip this cell and set the paths to data/075_images.npy

from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# ====================================
# Load the Clock dataset (75x75)
# ====================================
images_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_images.npy'
labels_path = '/content/drive/MyDrive/DL_Assignment_1/data/075_labels.npy'

images_np = np.load(images_path)
labels_np = np.load(labels_path)

#Check out our images
    #No longer needed at this point

#Convert into pytorch tensors
images = torch.from_numpy(images_np).float() #turn numpy array into pytorch tensor, float
images = images.unsqueeze(1) #add channel dimension, we need format (batch_size, channels, height, width) (greyscale = 1 channel)
mean = images.mean()
std = images.std()
```

```
images = (images - mean) / std #standardize the pixels (easier for training)

labels_hours = torch.from_numpy(labels_np[:, 0] + labels_np[:, 1] / 60.0).float() #NEW: mapping "03:00" → 3.0 and "05:30" → 5.5).
print(labels_hours.shape, labels_hours.min(), labels_hours.max()) #18'000 labels, 1 dimension (values go from 0 to 11.98).

#What's new:
#label_minutes is now a float number (e.g. 3.5 hours)
```

```
torch.Size([18000]) tensor(0.) tensor(11.9833)
```

```
#====================================
# Train-Validation-Test split
# ====================================
reset_seed()

dataset = TensorDataset(images, labels_hours)
n = len(dataset)

train_size = int(0.8 * n)
valid_size = int(0.1 * n)
test_size = n - train_size - valid_size

train_data, valid_data, test_data = random_split(
    dataset,
    [train_size, valid_size, test_size],
    generator=generator
)

batch_size = 64

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    shuffle=True,
    generator=generator
)

valid_dataloader = DataLoader(
    valid_data,
    batch_size=batch_size,
    shuffle=True
)

test_dataloader = DataLoader(
    test_data,
    batch_size=batch_size,
    shuffle=True
)
```

```
# ====================================
# Setting up the CNN for regression
# ====================================

class Net_4conv(nn.Module):   #Final CNN
    def __init__(self):  #allow to provide a desired number of classes
        super().__init__()
        #Input image (1, 75, 75) (greyscale, 75x75)
        self.conv1 = nn.Conv2d(1, 6, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3, padding=1)
        self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv4 = nn.Conv2d(32, 64, 3, padding=1)
        self.fc1 = nn.Linear(64 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 1) #NEW: now we want only 1 output value

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = self.pool(F.relu(self.conv4(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

#What's new?
#Now our NN needs to output one number, not n (num_classes) classes.
```

```
# ======================================================
# Defining the functions for training and evaluation
# ======================================================


#Train the model

def train_model(model, train_dataloader, optimizer, criterion, epochs): #NEW: no more bin_size

    """Train a model for a given number of epochs."""

    for epoch in range(epochs):

        model.train()
        running_loss = 0.0

        for inputs, hours in train_dataloader:

            #data agumentation
            #inputs = train_transform(inputs)

            #To use external GPU
            inputs = inputs.to(device)
            hours = hours.to(device)

            #zero the parameter gradients
            optimizer.zero_grad()

            #forward + backward + optimize
            outputs = model(inputs).squeeze(1) #NEW: these are the new predictions
            loss = criterion(outputs, hours) #NEW: regression loss
            loss.backward() #compute gradients
            optimizer.step() #optimize weights

            #compute running loss
            running_loss += loss.item()
        print(
            f"Epoch {epoch + 1}/{epochs}: "
            f"- Loss = {running_loss/len(train_dataloader):.4f}"
        )

#Validate the model

def evaluate_model(model, dataloader): #NEW: no more bin_size

    """Evaluate a model using the average circular time error."""

    model.eval()

    total_error = 0
    total = 0

    with torch.no_grad():
        for inputs, hours in dataloader:

            #To use external GPU
            inputs = inputs.to(device)
            hours = hours.to(device)

            #Get predictions (in total number of minutes)
            outputs = model(inputs).squeeze(1) #NEW: predictions (in hours, 3.5!!!)
            predicted_hours = torch.remainder(outputs, 12.0) #NEW: restrict prediction in 12 hours range

            #Calculate absolute difference and the circular difference (in minutes!!)
            predicted_minutes = predicted_hours * 60.0 #We re-convert them in minutes to make it comparable with the other models
            true_minutes = hours * 60.0 #It doesn't affect the learning process
            difference = torch.abs(predicted_minutes - true_minutes)
            circular_difference = torch.minimum(difference, 720 - difference)

            #Compute the errors
            total_error += circular_difference.sum().item()
            total += hours.size(0)

    common_sense_error = total_error / total

    return common_sense_error
```

```
# ====================================
# Experiment with different Loss Functions
# ====================================
```

```
#Loss function
loss_functions = {
    "MSELoss": nn.MSELoss(),
    "L1Loss": nn.L1Loss(),
    "SmoothL1Loss": nn.SmoothL1Loss(),
}
epochs = 20
learning_rate = 0.001
results, models = [], {}

for loss_name, criterion in loss_functions.items():

    print(f"\nTraining with {loss_name}")

    #New seed for every experiment
    reset_seed()

    #New model for each experiment
    model = Net_4conv().to(device)

    #Optimizer
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    #Train the model
    train_model(
        model,
        train_dataloader,
        optimizer,
        criterion,
        epochs
    )

    #Validate the model
    valid_error = evaluate_model(
        model,
        valid_dataloader
    )

    models[loss_name] = model

    #Store results
    results.append({
        "Loss function": loss_name,
        "Validation common-sense error (min)": valid_error
    })
    pd.DataFrame(results).to_csv("results_regress.csv", index=False)
    print(f"{loss_name} validation error: {valid_error:.2f} minutes")

results_df = pd.DataFrame(results)
print(results_df)
```

```
Epoch 14/20: - Loss = 3.6881
Epoch 15/20: - Loss = 3.3847
Epoch 16/20: - Loss = 3.0520
Epoch 17/20: - Loss = 2.6846
Epoch 18/20: - Loss = 2.5641
Epoch 19/20: - Loss = 2.2773
Epoch 20/20: - Loss = 2.0510
MSELoss validation error: 72.36 minutes

Training with L1Loss
Epoch 1/20: - Loss = 3.1415
Epoch 2/20: - Loss = 3.0182
Epoch 3/20: - Loss = 2.9858
Epoch 4/20: - Loss = 2.7983
Epoch 5/20: - Loss = 2.4413
Epoch 6/20: - Loss = 2.2306
Epoch 7/20: - Loss = 2.0374
Epoch 8/20: - Loss = 1.8959
```



```
Epoch 1/20: - Loss = 2.6644
Epoch 2/20: - Loss = 2.5479
Epoch 3/20: - Loss = 2.5142
Epoch 4/20: - Loss = 2.3036
Epoch 5/20: - Loss = 1.9667
Epoch 6/20: - Loss = 1.7264
Epoch 7/20: - Loss = 1.5408
Epoch 8/20: - Loss = 1.3852
Epoch 9/20: - Loss = 1.2161
Epoch 10/20: - Loss = 1.1056
Epoch 11/20: - Loss = 1.0074
Epoch 12/20: - Loss = 0.9242
Epoch 13/20: - Loss = 0.8548
Epoch 14/20: - Loss = 0.7854
Epoch 15/20: - Loss = 0.7371
Epoch 16/20: - Loss = 0.6803
Epoch 17/20: - Loss = 0.6258
Epoch 18/20: - Loss = 0.6017
Epoch 19/20: - Loss = 0.5638
Epoch 20/20: - Loss = 0.5361
SmoothL1Loss validation error: 59.70 minutes
  Loss function  Validation common-sense error (min)
0       MSELoss                             72.358219
1        L1Loss                             66.404196
2  SmoothL1Loss                             59.704752
```

```
# ====================================
# Test set - With Best Parameters
# ====================================

#Best Loss
best_loss = results_df.loc[results_df["Validation common-sense error (min)"].idxmin(), "Loss function"]

#Evaluate the final model on test set
test_error = evaluate_model(
    models[best_loss],
    test_dataloader
)

print(f"Best loss: {best_loss}, test error: {test_error:.2f} minutes")
```

```
Best loss: SmoothL1Loss, test error: 65.20 minutes
```

```
# ====================================
# Save Results
# ====================================

final = pd.DataFrame([{
    "Formulation": "Regression",
    "Setting": best_loss,
    "Epochs": epochs,
    "Validation error (min)": results_df["Validation common-sense error (min)"].min(),
    "Test error (min)": test_error
}])

print(final)
```

```
  Formulation       Setting  Epochs  Validation error (min)  Test error (min)
0  Regression  SmoothL1Loss      20               59.704752          65.20335
```